

Multimodal Large Model + Robotic Arm Sorting

Function Description

After the program runs, input the color block sorting order through the terminal. The large model will plan the steps to complete the color block sorting task, and then the program will control the robotic arm to sort the color blocks in order and place them at the designated positions.

Startup

Open the terminal and enter the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and enter the following command:

```
ros2 run text_chat text_chat
```

Then input the sorting order for the color blocks in the terminal. You can refer to the following examples:

```
Sort the color blocks on the desktop in the order of yellow, red, green
```

```
root@yahboom:~/# ros2 run text_chat text_chat
user input: Sort the color blocks on the desktop in the order of yellow, red, green
okay@, let me think for a moment... \[INFO] [1775121154.928760886] [text_chat_node]: Decision making AI planning:1. Call 'seewhat()' to observe the environment and determine the positions of the yellow, red, and green blocks.
2. Grasp the yellow block using 'grasp_obj(x1, y1, x2, y2)' based on its observed coordinates.
3. Move the yellow block to the desired position.
4. Call 'seewhat()' again to observe the environment and determine the position of the red block.
5. Grasp the red block using 'grasp_obj(x1, y1, x2, y2)' based on its observed coordinates.
6. Move the red block to the side of the yellow block.
7. Call 'seewhat()' again to observe the environment and determine the position of the green block.
8. Grasp the green block using 'grasp_obj(x1, y1, x2, y2)' based on its observed coordinates.
9. Move the green block to the side of the red block.
[INFO] [1775121158.964494862] [text_chat_node]: "action": ['seewhat()'], "response": Got it! I'm going to sort the color blocks in the order of yellow, red, and green. Let me start by taking a look at the current arrangement!
[INFO] [1775121166.299799587] [text_chat_node]: "action": ['grasp_obj(190, 300, 340, 450)'], "response": I see the blocks now! The yellow block is in the middle, the red one is on the right, and the green one is on the left. Let me grab the yellow block first!
[INFO] [1775121201.407148805] [text_chat_node]: "action": ["change_pose(190, 300, 340, 450, 'yellow', 'yellow', 5, 5)"], "response": Got the yellow block! Now, I'll move it to the desired position. Let me place it first in the sequence!
[INFO] [1775121224.394055410] [text_chat_node]: "action": ['seewhat()'], "response": Yellow block is now in place! Next up, I'll check where the red block is so I can move it next to the yellow one. Let me take a quick look!
[INFO] [1775121232.393160962] [text_chat_node]: "action": ['grasp_obj(380, 230, 540, 390)'], "response": I see the red block now! It's on the right side. Let me grab it and place it next to the yellow block.
[INFO] [1775121269.739729422] [text_chat_node]: "action": ["change_pose(380, 230, 540, 390, 'red', 'yellow', 4, 5)"], "response": Got the red block! Now, I'll move it to the side of the yellow block to continue the sequence. Let's do this!
[INFO] [1775121291.439593772] [text_chat_node]: "action": ['seewhat()'], "response": Red block is now next to the yellow one! Only one more to go—the green block. Let me check its position again!
[INFO] [1775121298.965077664] [text_chat_node]: "action": ['grasp_obj(140, 150, 280, 290)'], "response": I see the green block now! It's on the left side. Let me grab it and place it next to the red block to complete the sequence.
[INFO] [1775121334.451597016] [text_chat_node]: "action": ["change_pose(140, 150, 280, 290, 'green', 'red', 4, 5)"], "response": Got the green block! Now, I'll move it to the side of the red block to finish sorting them in the order: yellow, red, green. Let's do it!
[INFO] [1775121357.907383977] [text_chat_node]: "action": ['finishtask()'], "response": All done! The color blocks are now sorted in the order of yellow, red, and green. I hope you're happy with the result!
█
```

As shown in the figure above, before sorting the yellow color block, first check if it is in the removal list. If it is, then directly grip it from the removal list and place it at the yellow color block placement point; if not, then determine if there are other color blocks on top of the yellow color block. If there are, first remove the other color blocks above the yellow color block. If there are none, then directly grip it and place it at the yellow color block placement point. Other color blocks are also sorted according to this execution order until the task is completed.

Task Planning

1. Call `check_remove(color)` function to check if the current target color block to be sorted exists in the removal list (where `color` is the color of the current target color block to be sorted, value range is `red`, `blue`, `green` or `yellow`);
2. If the current target color block to be sorted exists in the removal list, execute the following operations:
 - (1) Call `grasp_from_rm_list(color)` function to grip the current target color block to be sorted from

the removal list (meaning of `color` parameter is the same as above);

(2) Call `arm_stack(color)` to place the target color block at the specified position (where `color` parameter meaning is the same as above).

3. If the target color block does not exist in the removal list, follow these steps to sort the current color block to be sorted (only output one action function at a time):

(1) Call `seewhat()` function to find the current target color block to be sorted;

(2) Call `seewhat()` function to determine if there are other objects on top of the current target color block to be sorted;

(3) If there are no other objects on top of the current target color block to be sorted, execute the following steps (only output one action function at a time):

① Call `grasp_obj(x1, y1, x2, y2)` function to grip the current target color block to be sorted (where `(x1, y1, x2, y2)` are the top surface border coordinates of the current target color block to be sorted);

② Call `arm_stack(color)` to place the current target color block to be sorted at the specified position (where `color` parameter meaning is the same as above).

(4) If there are other objects on top of the current target color block to be sorted, execute the following steps to first remove that object then sort the current target color block to be sorted, here only one action function can be output at a time:

① Call `remove_obj(x1, y1, x2, y2, color)` function to remove that object (where `(x1, y1, x2, y2)` are the top surface border coordinates of that object, `color` is the color of that object, value range is `red, blue, green or yellow`);

② Call `seewhat()` function to observe the environment;

③ If there are no other objects on top of the current target color block to be sorted, call `grasp_obj(x1, y1, x2, y2)` function to grip the current target color block to be sorted (where `(x1, y1, x2, y2)` are the top surface border coordinates of the current target color block to be sorted);

④ Call `arm_stack(color)` to place the target color block at the specified position (where `color` parameter meaning is the same as above).

Core Code Analysis

check_remove(color) Function

Source code path: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```
def check_remove(self,color):
    #Get the current color parameter
    check_color = color.strip("\'")
    #If the current color is in the self.cur_rm_pose dictionary, it is in the removal list;
    #otherwise, it is not found in the removal list
    if check_color in self.cur_rm_pose:
        self.action_status_pub("color is in the rm_list")
    else:
        self.action_status_pub("color is not in the rm_list")
    #You can see what the large model receives for "color is in the rm_list" and "color is not
    #in the rm_list" respectively
    "color is in the rm_list" : "The color block exists in the removal list, grip the color
    block from the removal list",
    "color is not in the rm_list" : "The color block does not exist in the removal list,
    determine if there are other objects on top of the color block"
```

grasp_from_rm_list Function

Source code path: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```
def grasp_from_rm_list(self,color):
    #Get the current color parameter
    tar_color = color.strip("\")
    #According to the current color parameter, find the corresponding key value in the
    dictionary and assign it to tar_joints
    tar_joints = self.cur_rm_pose.get(tar_color)
    #Control the six servos of the robotic arm
    Arm.Arm_serial_servo_write6(tar_joints[0], tar_joints[1], tar_joints[2], tar_joints[3],
    tar_joints[4], tar_joints[5],2000)
    time.sleep(2.0)
    Arm.Arm_serial_servo_write(6, 140, 1000)
    time.sleep(2.0)
    Arm.Arm_serial_servo_write6(90,120,10,10,90,140,2000)
    time.sleep(2.0)
    #Feedback to the large model that gripping the color block from the removal list is
    complete
    self.action_status_pub("grasp_from_rm_list_done")
```

arm_stack Function

Source code path: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```
def arm_stack(self,color):
    #Get the current color parameter
    tar_color = color.strip("\")
    #According to the current color parameter, select the placement pose of the robotic arm
    if tar_color == 'red':
        Arm.Arm_serial_servo_write6(117, 19, 66, 56, 90,self.grasp_joint,2000)
        time.sleep(2.0)
    elif tar_color == 'green':
        Arm.Arm_serial_servo_write6(136, 66, 20, 29, 90,self.grasp_joint,2000)
        time.sleep(2.0)
    elif tar_color == 'blue':
        Arm.Arm_serial_servo_write6(44, 66, 20, 28, 90,self.grasp_joint,2000)
        time.sleep(2.0)
    elif tar_color == 'yellow':
        Arm.Arm_serial_servo_write6(65, 22, 64, 56, 90,self.grasp_joint,2000)
        time.sleep(2.0)
    Arm.Arm_serial_servo_write(6, 30, 1500)
    time.sleep(1.5)
    Arm.Arm_serial_servo_write6(90,120,10,10,90,30,2000)
    time.sleep(2.0)
    #Feedback to the large model that placing the color block is complete
    self.action_status_pub("arm_stack_done", color=color)
```

remove_obj Function

Source code path: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```
def remove_obj(self, x1, y1, x2, y2,color):
    #Get the current color parameter
    cur_rm_name = color.strip('"\'') # Remove single and double quotes
    self.cur_rm_name = cur_rm_name
    #Create a key in the dictionary
    self.cur_rm_pose[self.cur_rm_name] = []
    self.remove_cnt = -self.remove_cnt
    """
    Grip object
    x1,y1,x2,y2: Outer frame coordinates of the object
    """

    #Object gripping program
    cmd1 = "ros2 run largemodel_arm KCF_Grap_Move"
    #KCF tracking positioning program
    cmd2 = "ros2 run largemodel_arm ALM_KCF_Tracker"
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=ALM_KCF_Tracker",
            "--",
            "bash",
            "-c",
            f"{cmd2}; exec bash",
        ]
    )
    time.sleep(5.0) #wait for ALM_KCF_Tracker to start
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=grasp_desktop",
            "--",
            "bash",
            "-c",
            f"{cmd1}; exec bash",
        ]
    )
    time.sleep(2.0)
    if self.stack_flag == True:
        self.get_logger().info('Publish the stack_step topic...')
        step_ = Int16()
        step_.data = self.step
        self.step_pub.publish(step_)
        self.step = self.step + 1
    #Publish outer frame coordinate information topic data, which will be subscribed to by
    the ALM_KCF_Tracker node
    x1 = int(x1)
    y1 = int(y1)
    x2 = int(x2)
    y2 = int(y2)
```

```

self.object_position_pub.publish(Int16MultiArray(data=[x1, y1, x2, y2]))

while not self.grasp_obj_future.done():
    if self.interrupt_flag:
        self.check_close_grasp_obj()
        #self.pubSix_Arm(self.init_joints) # Retract the robotic arm
        #Arm.Arm_serial_servo_write6(90,150,12,20,90,30,1000)
        self.stop()
        return
    time.sleep(0.1)
self.check_close_remove_obj()

tmp_joint1 = 90 + self.remove_cnt*60 #30 150
#Add the calculated joint value for placing the removed color block to the dictionary as
the key value of self.cur_rm_pose[self.cur_rm_name]
self.cur_rm_pose[self.cur_rm_name] = [tmp_joint1,50,40,20,90,30]
self.get_logger().info(f"self.cur_rm_pose:{self.cur_rm_pose}")
#Control the robotic arm to move to the specified pose to place the removed color block
Arm.Arm_serial_servo_write6(tmp_joint1,50,40,20,90,140,2000)
time.sleep(2.0)
Arm.Arm_serial_servo_write6(tmp_joint1,50,40,20,90,30,2000)
time.sleep(2.0)
Arm.Arm_serial_servo_write6(90,120,10,10,90,30,2000)
time.sleep(2.0)
#Feedback to the large model that removing the color block is complete
self.action_status_pub("remove_obj_done", x1=x1, y1=y1, x2=x2, y2=y2, color=color)

```

The KCF_Grap_Move and ALM_KCF_Tracker nodes have been analyzed in previous content. You can refer to the content in [AI Model - Text Version] - [Multimodal Large Model + Robotic Arm Gripping] section 2.4.3.

grasp_obj(x1, y1, x2, y2) Function

Source code path: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

The grasp_obj(x1, y1, x2, y2) function has been analyzed in previous content. You can refer to the content in [AI Model - Text Version] - [Multimodal Large Model + Robotic Arm Gripping] section 2.4.3.